

What is a “window manager”

Window management is something everyone is familiar with. Every desktop has one. Most are floating, some are tiling, other are scrolling, but what’s often more difficult to see, is the distinction between *window management* and the remaining requirements for the typical desktop stack. There are 5 main parts to any computer desktop:

1. **Kernel, OS and hardware** - Responsible for receiving and forwarding input events, and drawing to outputs.
2. **Display server** - Coordinates dataflow between desktop clients and other systems, including the kernel and hardware.
3. **Window manager** - Entirely responsible for the arrangement of windows and windowing patterns.
4. **Compositor** - Takes all client surfaces and their transforms from the window manager to composite the final rendered buffer.
5. **Clients** - The applications that users run, who get a single buffer to render to and nothing more.

The window manager itself plays only a part in the desktop experience, but throughout time, the window manager has been consistently coupled with other unrelated systems. Historically, X11 coupled the window manager with the display server, then called to an external compositor for a final image. In more recent times, the typical Wayland stack couples the display server, compositor, and window manager into one wayland compositor¹. Neither of these approaches are perfect, but the common denominator is that the window manager is never independent. This makes the entire process of creating a window manager far more challenging than it needs to be, when the responsibilities of a window manager are actually quite simple.

Existing solutions

There do exist projects that give the window management control to the user. Within X11, DWM and XMonad are two of these projects. DWM is written and configured in C by modifying the source code of the project directly. This is not approachable at all to the average user whatsoever, which is where XMonad comes in. It similarly is written and configured in the same language, that being Haskell. While it utilizes Haskell language features, such as typeclasses, making configuration easier, simple windowing patterns are still very long winded. Both of these wonderful projects unfortunately suffer from being too programmable to the point where they are daunting and unapproachable to prospective users.

Wayland projects exist trying to accomplish similar goals. During its infancy, River was a Wayland compositor in the spirit of DWM, following the typical structure of Wayland compositors. In recent years however, Isaac Freund, the lead developer, has sought to completely banish the responsibility of window management from the project. Instead, River now implements a custom Wayland protocol used to interface with a properly external window manager, effectively making River a compositor and display server only. This protocol can be interfaced with in nearly limitless ways, but fails to escape the fact that simple windowing patterns now take even more technical complexity than they did on X11.

¹This is where common terminology conflicts with technical jargon. Within X11, the window manager and display server are collectively called the “window manager” while the compositor is left separated. Within Wayland, a “Wayland compositor” is actually a compositor, window manager, and display server all together.